

Structured-Prompt-Driven Development (SPDD)

Scaling AI Delivery with
First-Class Artifacts



The Paradox of AI Coding Speed

Local speed improves, but system throughput flatlines. Fast, ambiguous code generation creates a bottleneck of integration friction, inconsistent reviews, and unaligned output.

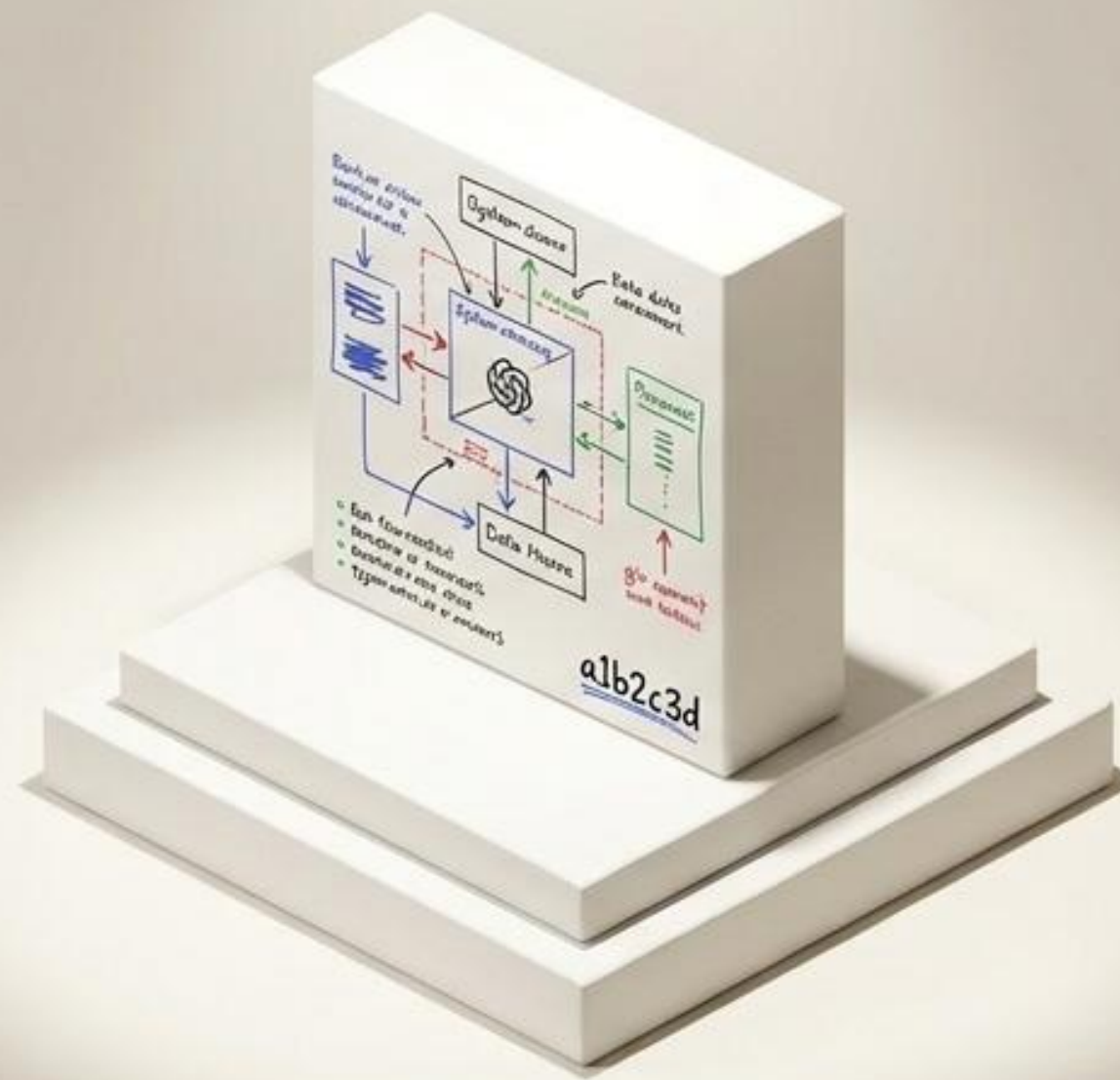


The Missing Delivery Artifact

To scale AI from personal efficiency to organizational capability, we must stop treating prompts as disposable chat logs. Prompts must become governed, version-controlled, and reusable first-class delivery artifacts.



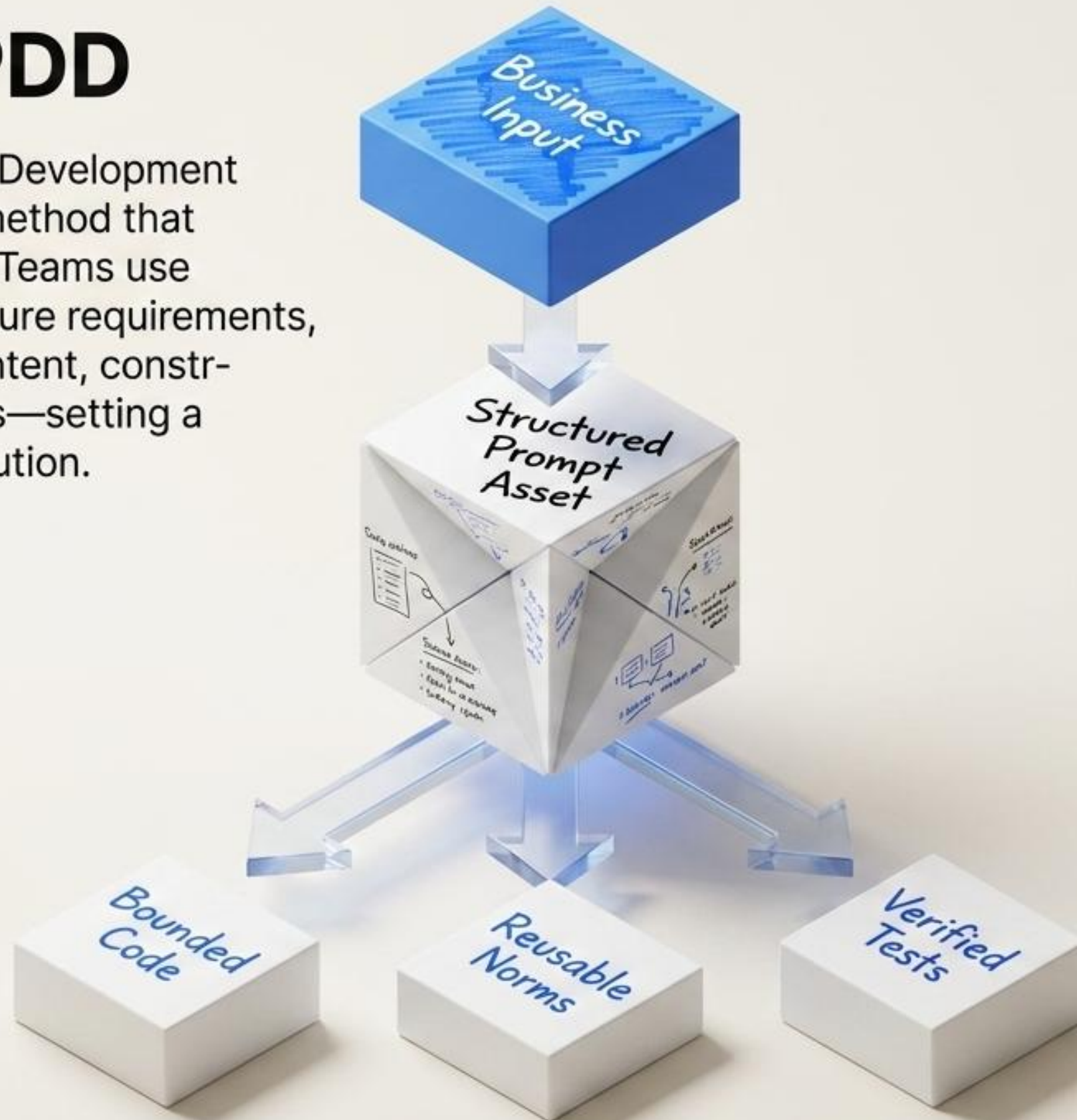
Ephemeral Chat



First-Class Artifact

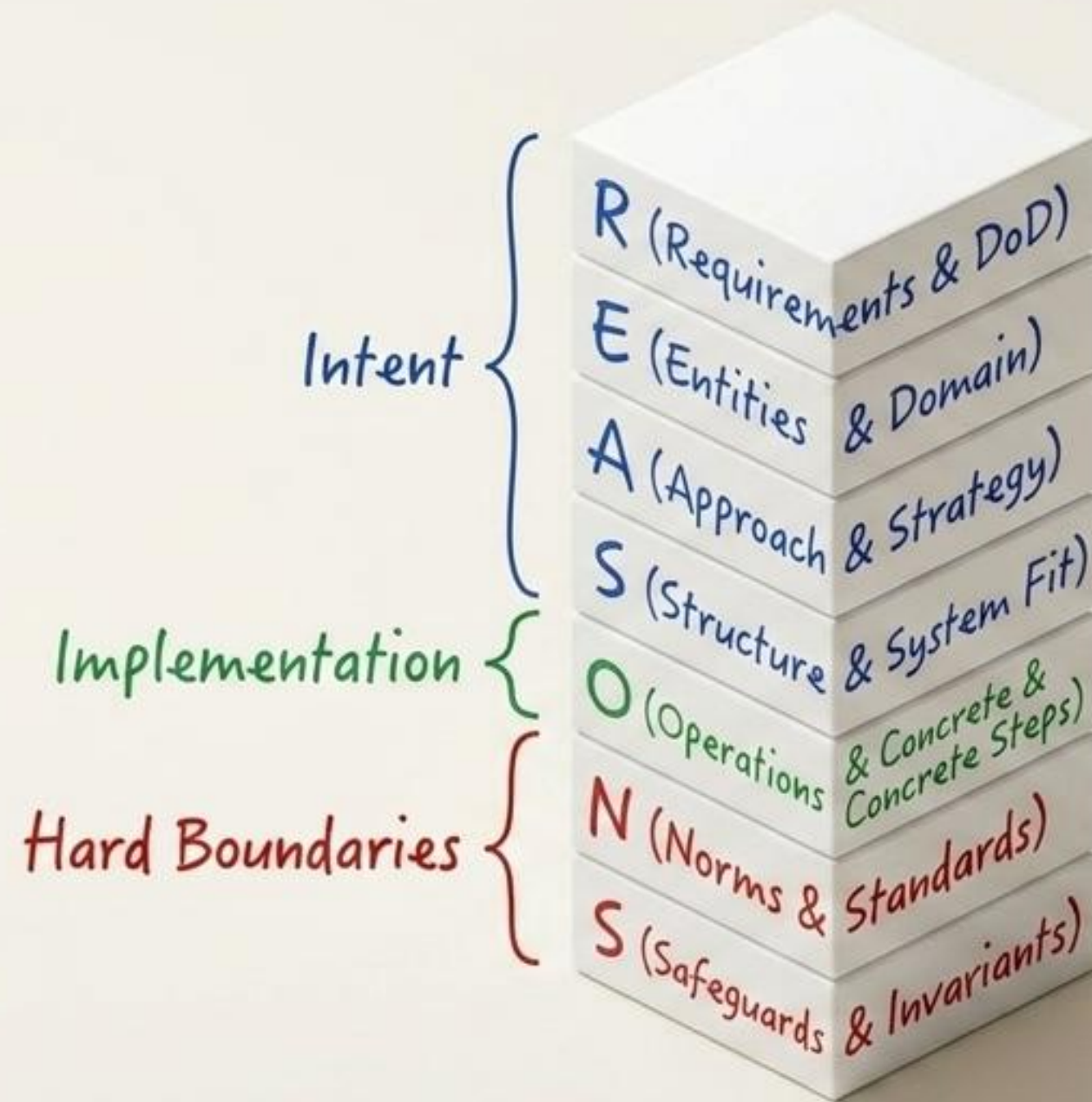
Defining SPDD

Structured-Prompt-Driven Development (SPDD) is an engineering method that forces clarity before code. Teams use structured prompts to capture requirements, domain language, design intent, constraints, and task breakdowns—setting a strict boundary for AI execution.



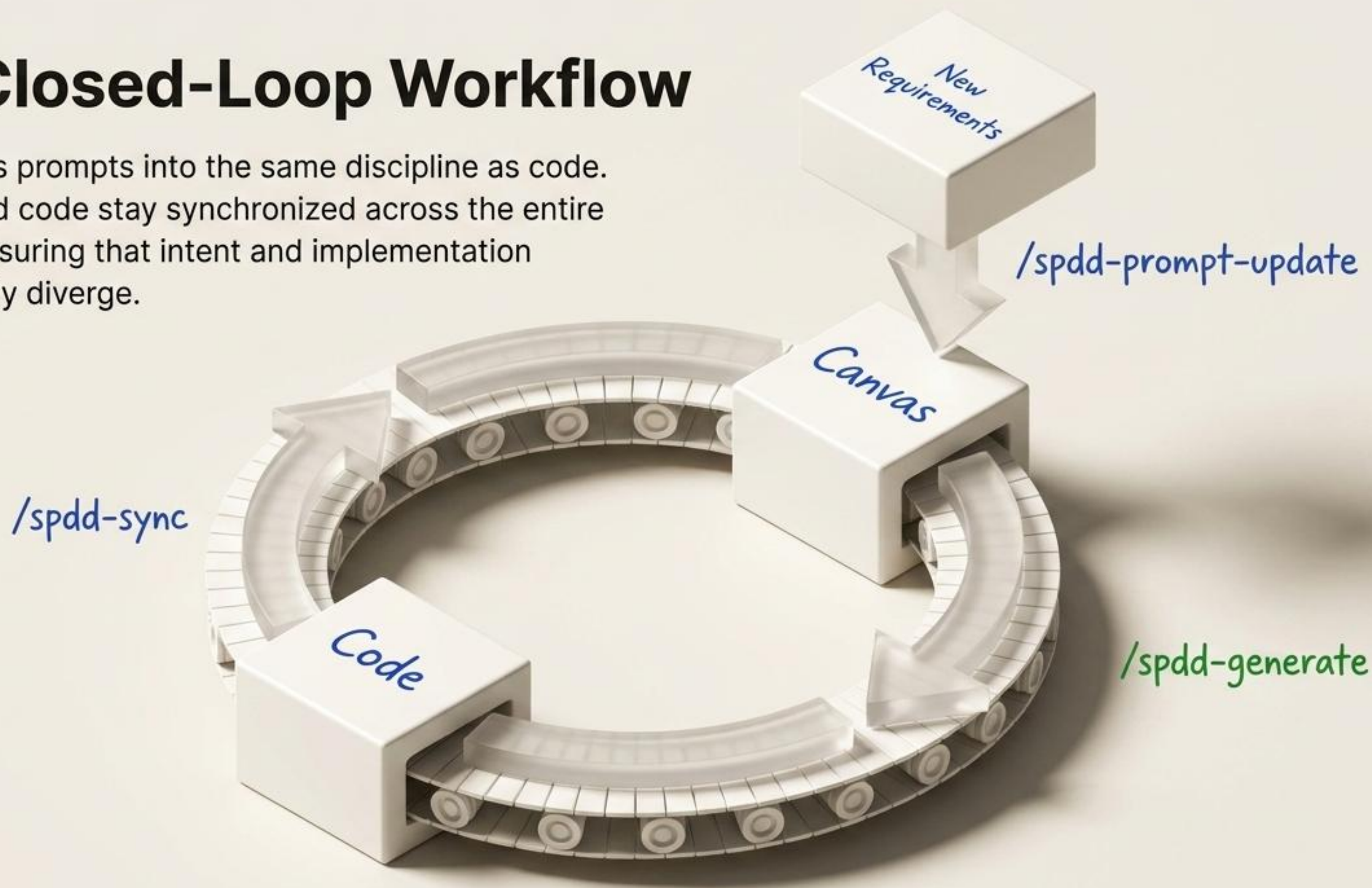
The REASONS Canvas: An Architectural Blueprint

A seven-part structure that guides a prompt from intent to design, execution, and governance.
The model is guided by explicit boundaries, not guesswork.



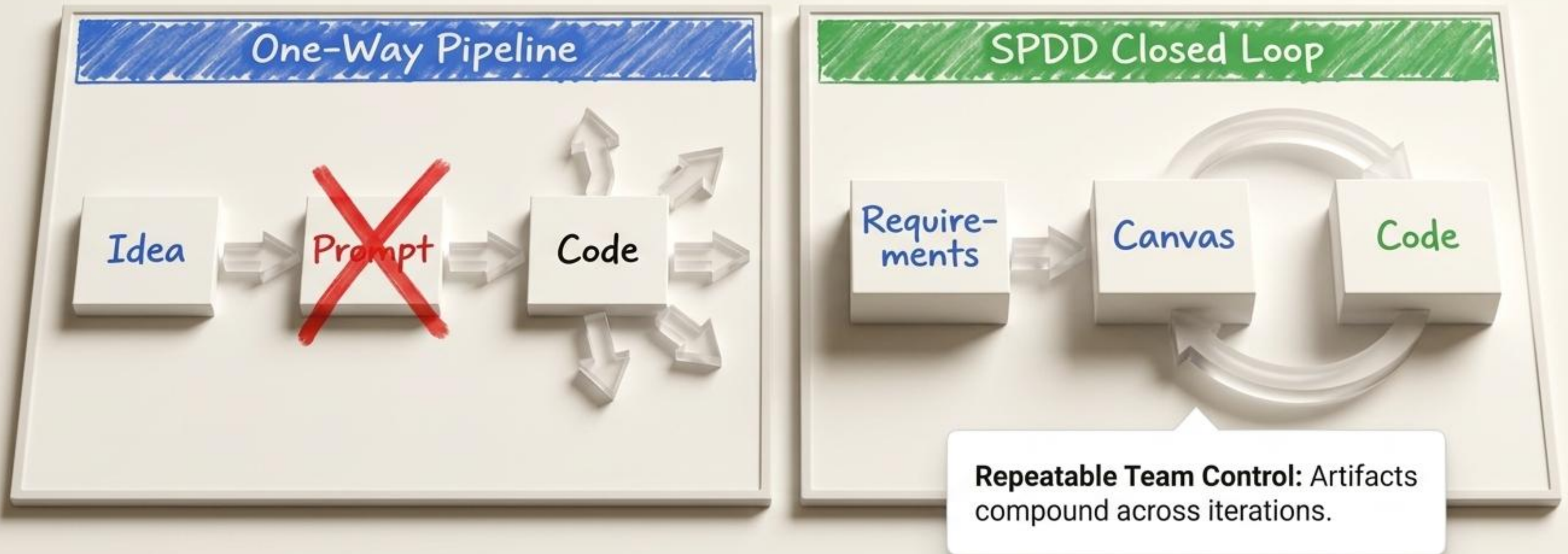
The Closed-Loop Workflow

SPDD brings prompts into the same discipline as code. Prompts and code stay synchronized across the entire lifecycle, ensuring that intent and implementation never silently diverge.



Spec-Driven vs. SPDD: The Governance Gap

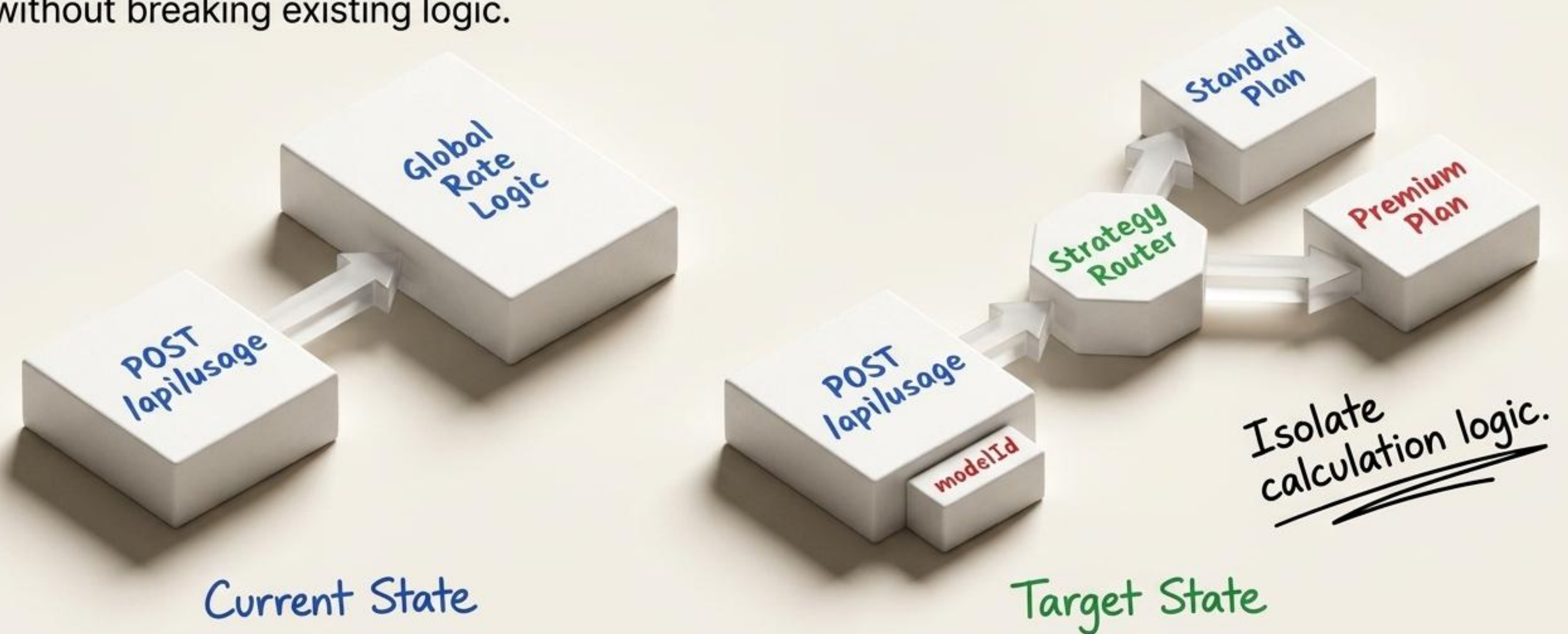
SPDD adds a method for how a spec is produced, reviewed, and permanently synchronized with the resulting codebase.



Case Study: Evolving the Billing Engine

Enhancing a static billing system to support multi-plan, model-aware pricing.

The goal: add dynamic routing for Standard (global quota) and Premium (split-rate) plans without breaking existing logic.



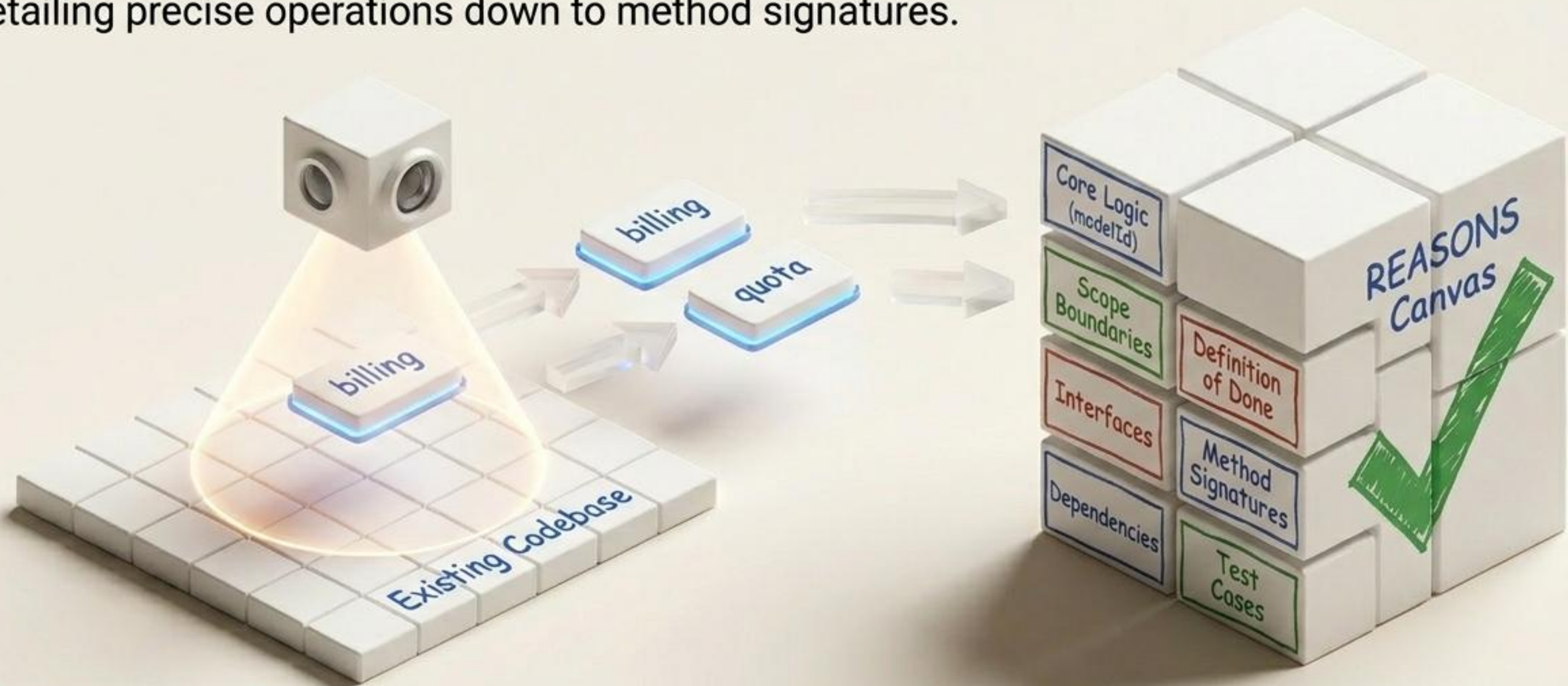
Step 1 & 2: Alignment and Clarification

We begin by breaking large requirements into independent INVEST stories (/spdd-story). The developer and product owner align on core logic, scope boundaries, and exact testable scenarios before any technical design begins.



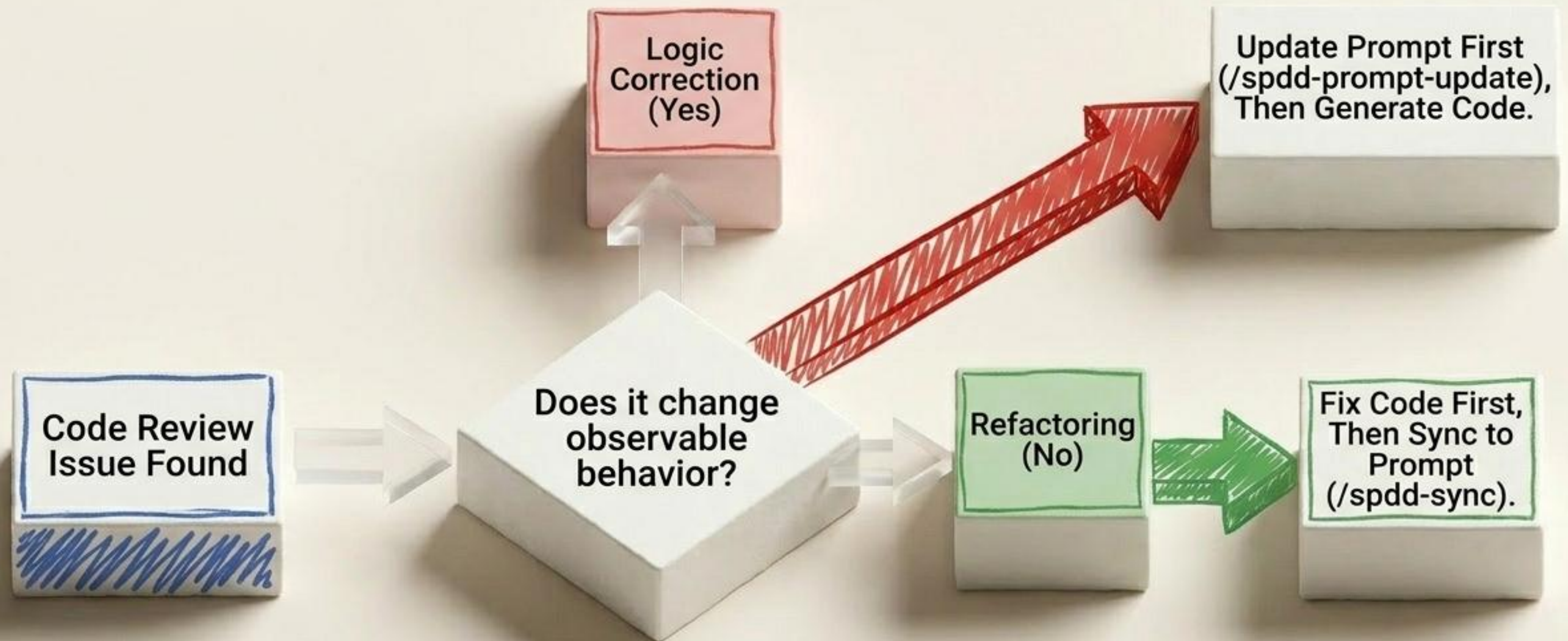
Step 3 & 4: Strategic Analysis and Canvas Generation

/spdd-analysis scans relevant code against the new story to map domain concepts and risks.
/spdd-reasons-canvas then translates this shared understanding into an executable blueprint, detailing precise operations down to method signatures.



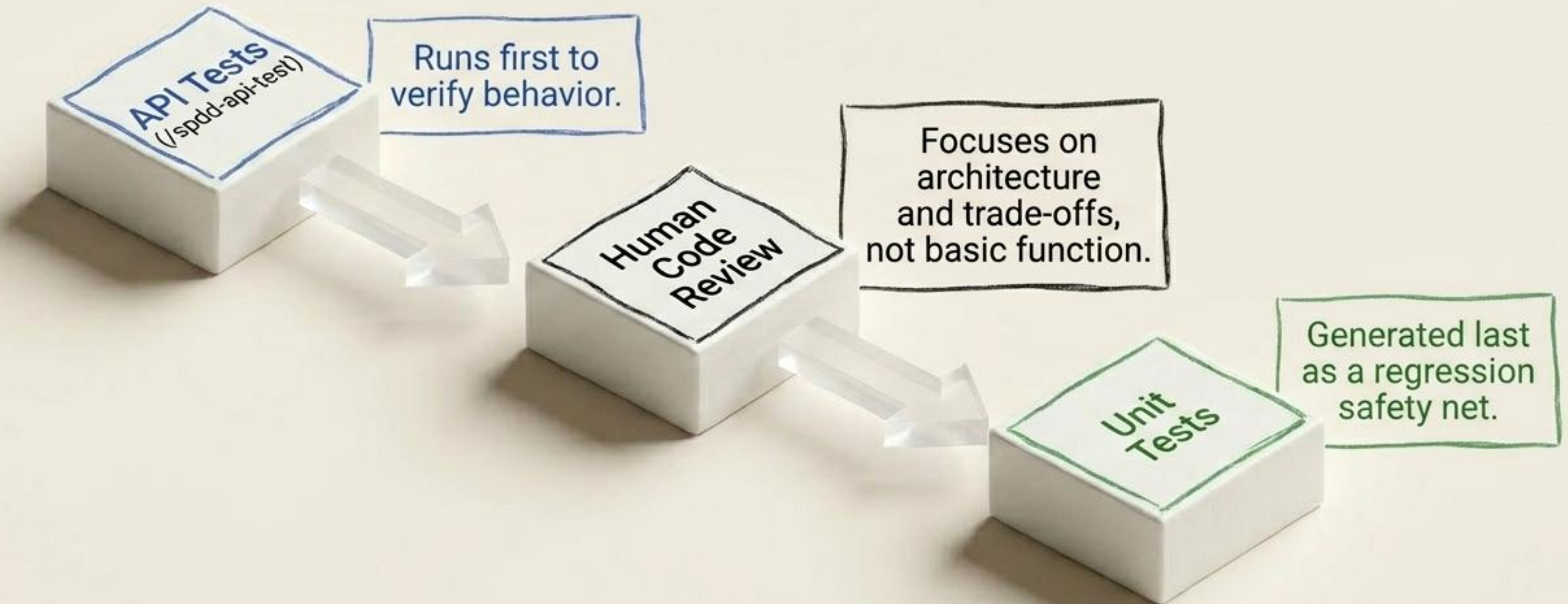
Step 5: The Golden Rule of Course Correction

When reality diverges from the generated code, how you fix it depends entirely on whether the system's observable behavior has changed.



Step 6: Validating the Implementation

Functional validation is distributed across the workflow to maximize human leverage. API tests validate the system boundaries cheaply before code review, while unit tests lock in the core logic after human alignment is finalized.



The Shifting Value of the Developer

SPDD is a material change in how software is built. In an AI-assisted world, developer value shifts away from raw typing speed and toward three meta-skills:



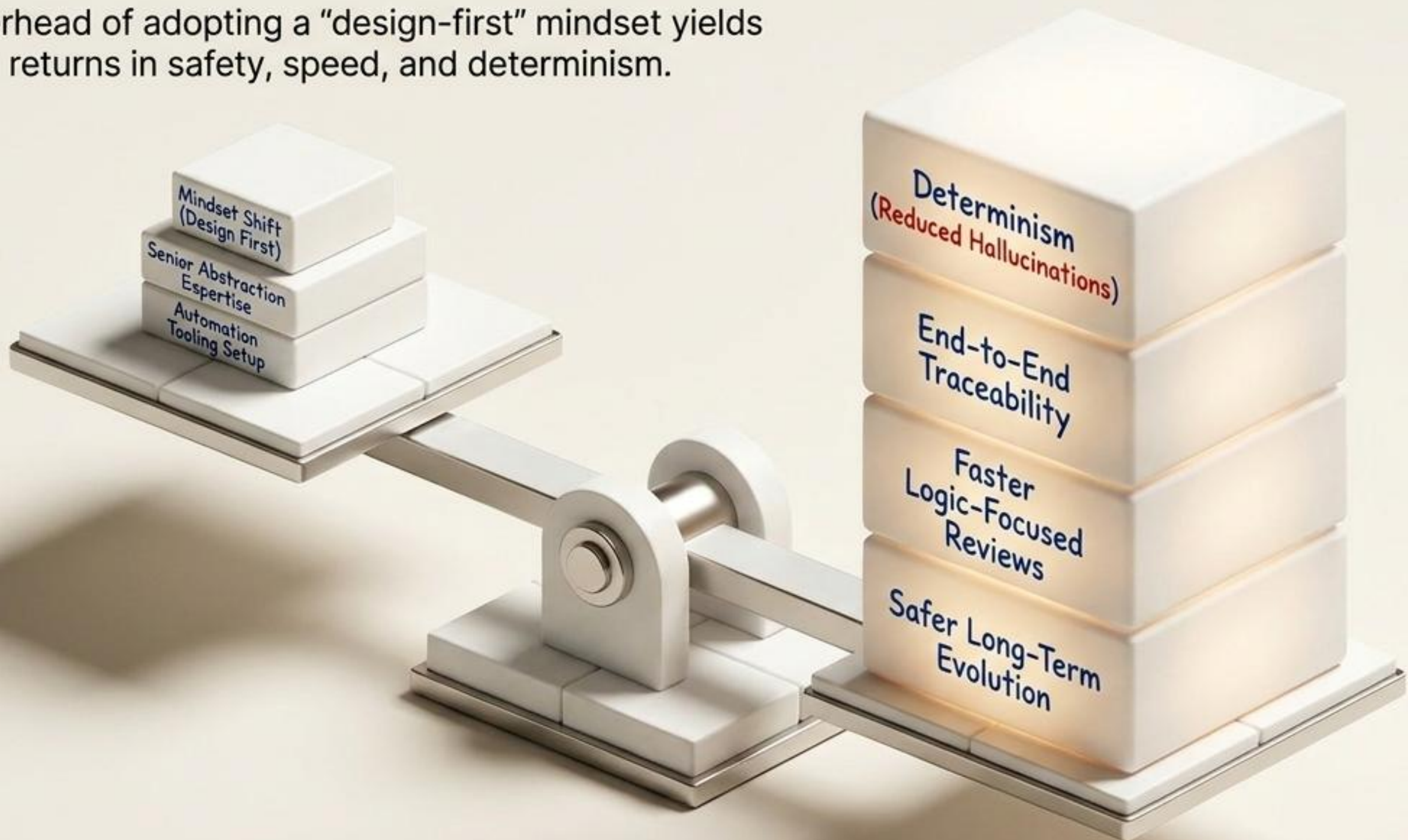
SPDD Fitness Assessment

SPDD is an engineering investment. It pays the highest dividends in logic-heavy domains and environments with strict constraints.



The ROI of Intentional Governance

The initial overhead of adopting a “design-first” mindset yields compounding returns in safety, speed, and determinism.



Lowering the Barrier: The Future of SPDD

Currently, SPDD requires a high bar of modeling and abstraction expertise. The roadmap focuses on shifting SPDD from a practice of individual craftsmanship to an automated, organization-level asset system with built-in decision memory and asset-layer verification.



Intentional Engineering in the AI Era

Software development is no longer a contest of typing speed or model IQ. It is a contest of engineer cognitive bandwidth—how clearly we can think, frame problems, and make decisions.

**“In science, if you know what you are doing, you shouldn’t be doing it.
In engineering, if you don’t know what you are doing, you shouldn’t be doing it.”**
— Richard W. Hamming

